Estructuras de datos en Python

Las estructuras del curso, con otro lenguaje y los mismos costos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Contenido

1. Repaso: heapsort y la cola de prioridad
2. `list`: el vector de Python
3. `deque`: colas y pilas
4. `dict` y `set`: tablas hash
5. `heapq`: el montículo de Python
6. Nodos en Python: la pila y la lista enlazada
7. Qué estructura elegir
8. Para practicar en casa
9. Cierre

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

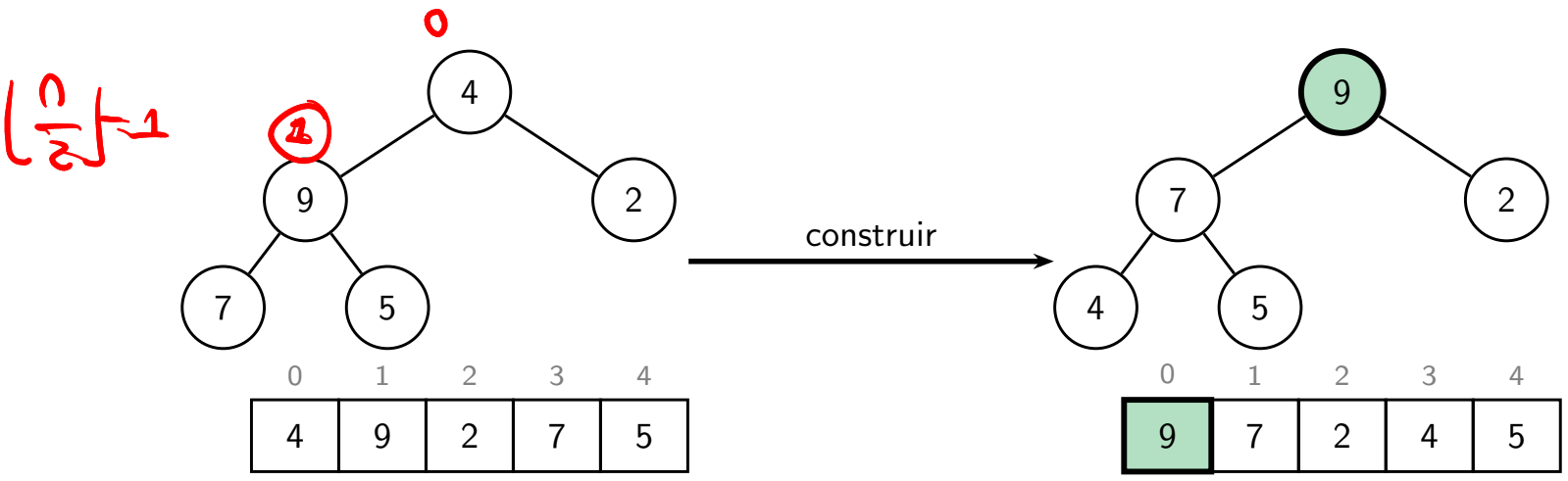
8 Para practicar en casa

9 Cierre

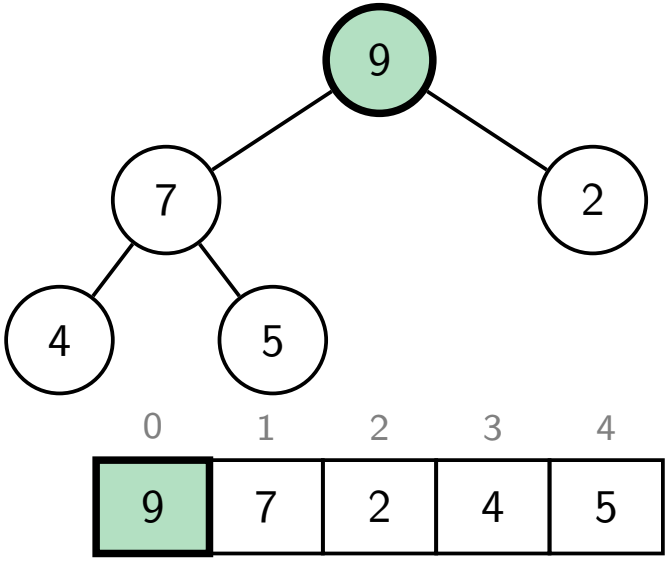
Heapsort en dos fases

Ordenar con un montículo (CLRS, §6.3 y §6.4)

Primero se construye el montículo de máximos, bajando cada casilla con hijos desde $\lfloor n/2 \rfloor - 1$ hasta la raíz: $\Theta(n)$. Después, $n-1$ veces, el máximo pasa al final y la nueva raíz baja: $\Theta(n \log n)$, sin arreglo auxiliar.



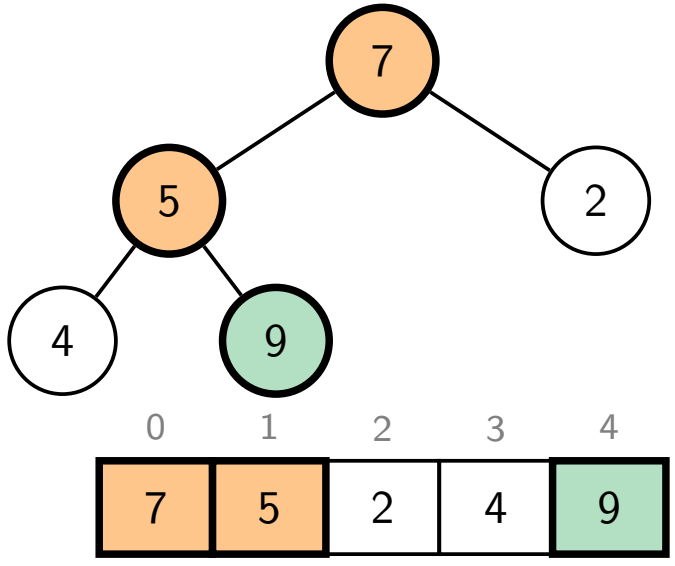
¿Qué deja la primera ronda?



Pregunta

Sobre el montículo 9 7 2 4 5, la primera ronda de heapsort intercambia `a[0]` con `a[4]` y baja la nueva raíz en las cuatro primeras casillas. ¿Cómo queda el arreglo?

¿Qué deja la primera ronda?



Respuesta

El 9 va a `a[4]` y queda en su sitio. El 5 sube a la raíz, se compara con 7 y 2 y baja hacia el 7; en `a[1]` su único hijo es el 4, y ahí se detiene: `7 5 2 4 | 9`.

El contenedor `priority_queue`

Un montículo de máximos sobre un `vector` (CLRS, §6.5)

`push` pone la clave al final y la sube, `pop` saca la de la raíz y baja la última, y `top` lee `a[0]`. Con `greater<int>` como comparador, arriba queda el menor.

Operación	Tiempo	Por qué
`push(x)`	$O(\log n)$	sube por un camino de altura $\lfloor \log_2 n \rfloor$
`pop()`	$O(\log n)$	la última baja por un camino
`top()`	$\Theta(1)$	lee la raíz

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

Una lista sin tipo

En C++:

```cpp
vector<int> notas;
notas.push_back(38);
notas.push_back(45);
notas.push_back(29);
```

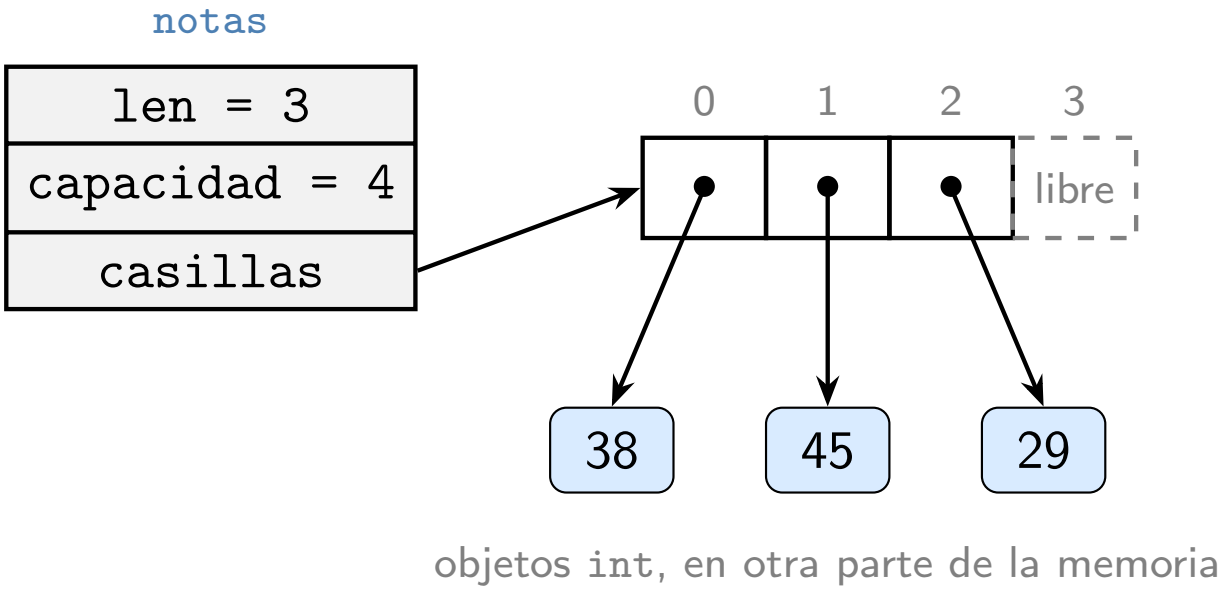
En Python:

```python
notas = [38, 45, 29]
```

Pregunta

En Python no se declara el tipo de `notas` ni de sus elementos. Si las casillas no saben qué guardan, ¿qué hay en cada una?

Una lista sin tipo

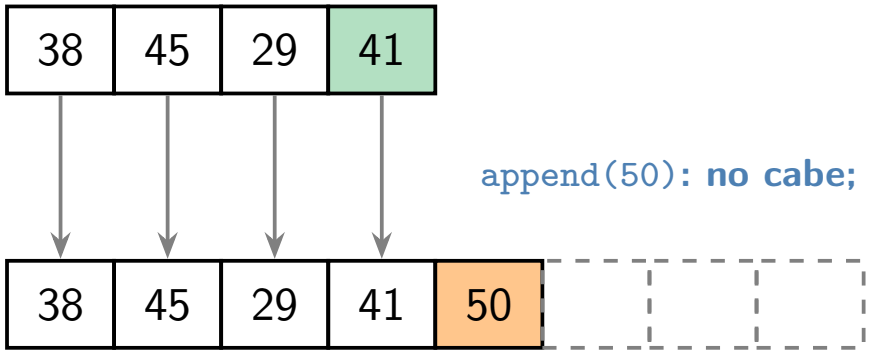


Un arreglo de referencias

Una `list` es un arreglo contiguo de referencias, con su tamaño y su capacidad al lado. Cada casilla apunta a un objeto, y por eso una lista puede guardar cualquier tipo. Acceder a `notas[i]` cuesta $\Theta(1)$, como en el `vector`.

Cuando el arreglo se llena

`append(41)`: **cabe en la casilla 3**



Crecer como el `vector`

Si no queda casilla libre, `append` pide un arreglo más grande y puede tener que copiar las referencias: esa llamada cuesta $O(n)$. Como el arreglo crece en proporción a su tamaño, la copia es rara, y n llamadas a `append` cuestan $O(n)$ en total: $O(1)$ amortizado (CLRS, §16.4).

Medir la capacidad

```
import sys

def capacidad(lista):
    vacia = sys.getsizeof([])
    return (sys.getsizeof(lista) -
    ↪  vacia) // 8

lista = []
anterior = -1
i = 0
while i < 20:
    c = capacidad(lista)
    if c != anterior:
        print("len", len(lista), "cap",
        ↪  c)
        anterior = c
    lista.append(i)
    i = i + 1
```

```
len 0 cap 0
len 1 cap 4
len 5 cap 8
len 9 cap 16
len 17 cap 24
```

Medido con CPython 3.14: la capacidad pasa por 4, 8, 16 y 24. En 20 llamadas a append, el arreglo se cambió 4 veces.

Las operaciones del `vector`, en Python

C++ (`vector<int> v`)	Python (`l = []`)
`v.push_back(x)`	`l.append(x)`
`v[i], v[i] = x`	`l[i], l[i] = x`
`v.back(); v.pop_back()`	`x = l.pop()`
`v.insert(v.begin() + i, x)`	`l.insert(i, x)`
`v.erase(v.begin() + i)`	`l.pop(i)`
`v.size()`	`len(l)`
`find(v.begin(), v.end(), x) != v.end()`	`x in l`
`sort(v.begin(), v.end())`	`l.sort()`

`l.pop()` quita el último y además lo devuelve; en la STL son dos pasos.

list en uso

```
notas = [38, 45, 29]
notas.append(41)          # como push_back
print(notas, len(notas))
print(notas[2])           # como v[2]
notas[0] = 40
notas.insert(1, 33)       # corre lo que
↪  sigue
print(notas)
ultima = notas.pop()      # quita el ultimo
print(ultima, notas)
primera = notas.pop(0)    # corre todos
print(primera, notas)
print(45 in notas, 50 in notas)
```

```
[38, 45, 29, 41] 4
29
[40, 33, 45, 29, 41]
41 [40, 33, 45, 29]
40 [33, 45, 29]
True False
```

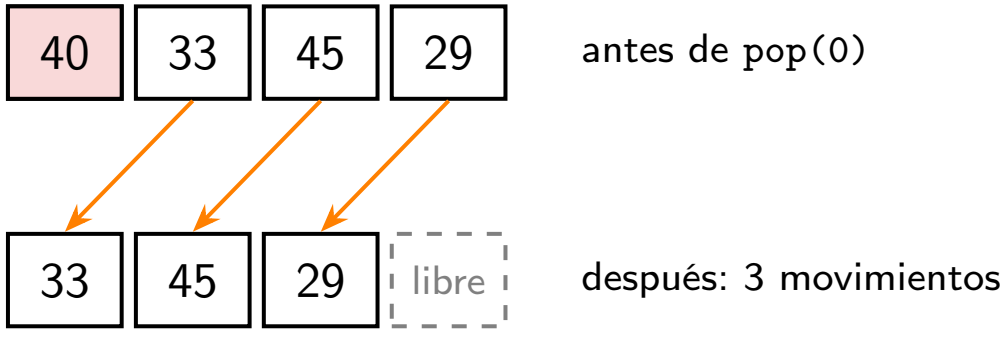
`uso_vector.cpp` hace lo mismo con `vector`, en las notas de la sesión.

¿Cuánto cuesta `pop(0)`?

Pregunta

Una lista tiene un millón de elementos. ¿Cuánto cuesta `l.pop()`? ¿Y `l.pop(0)`?

¿Cuánto cuesta `pop(0)`?



Respuesta

`l.pop()` quita la última casilla y no mueve nada: $\Theta(1)$. `l.pop(0)` corre todas las demás una casilla a la izquierda: $n - 1$ movimientos, $\Theta(n)$. Igual pasa con `insert(0, x)` y con `erase(v.begin())` en el `vector`.

Lo que cuesta cada operación de `list`

Operación	Tiempo	Por qué
`l[i]`, `l[i]=x`	$\Theta(1)$	la dirección de la casilla se calcula
`append(x)`	$O(1)$ amort.	casi siempre cabe; al crecer copia todo
`pop()`	$\Theta(1)$	quita la última, nada se corre
`insert(k,x)`, `pop(k)`	$O(n-k)$	se corren las casillas de la derecha
`x in l`	$O(n)$	compara de una en una
`len(l)`	$\Theta(1)$	el tamaño está guardado
`l.sort()`	$O(n \log n)$	ordenamiento por comparaciones

Espacio: $\Theta(n)$ referencias, con capacidad $\geq n$, más los objetos; $O(1)$ adicional por operación y hasta $O(n)$ en `sort`. Fuente: *Time complexity of operations on built-in types*.

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

La ventanilla de turnos

Problema

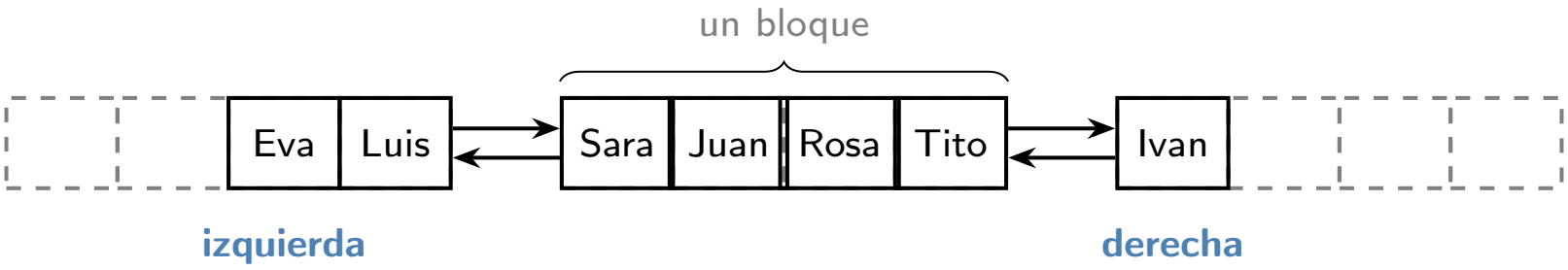
Una ventanilla atiende n personas en orden de llegada. Alguien propone una `list`: `append` al llegar y `pop(0)` al atender. ¿Cuánto cuesta atender a las n?

La ventanilla de turnos

Respuesta

Cada `pop(0)` corre lo que queda:
$(n-1)+(n-2)+\cdots+0 = n(n-1)/2$ movimientos, $\Theta(n^2)$. La cola necesita sacar por la izquierda en $\Theta(1)$, y eso lo da `collections.deque`, la cola doble de Python, como el `deque` de la STL.

El deque por dentro



Bloques enlazados por los dos lados

CPython guarda el deque en bloques de casillas, enlazados en una lista doble. Entrar o salir por un extremo toca solo el bloque de ese extremo, y cuando se llena se enlaza uno nuevo: $\Theta(1)$ por la izquierda y por la derecha.

La cola de turnos en los dos lenguajes

```python
from collections import deque

turnos = deque()
turnos.append("Ana")       # por
↪  la derecha
turnos.append("Luis")
turnos.append("Sara")
print(turnos[0], len(turnos))
atendido = turnos.popleft()
print("atiende a", atendido)
turnos.append("Juan")
turnos.appendleft("Eva")  # tiene
↪  cita
print(turnos)
```

```cpp
deque<string> t;
t.push_back("Ana");
t.push_back("Luis");
t.push_back("Sara");
printf("%s %d\n",
↪  t.front().c_str(),
       (int) t.size());
string atendido = t.front();
t.pop_front();
printf("atiende a %s\n",
       atendido.c_str());
t.push_back("Juan");
```

```
Ana 3
atiende a Ana
deque(['Eva', 'Luis', 'Sara', 'Juan'])
```

La pila: una `list` con la cima al final

```
acciones = []
acciones.append("escribir")
acciones.append("borrar")
acciones.append("pegar")
print("tope:", acciones[-1])
while len(acciones) > 0:
    print("deshace", acciones.pop())
```

```
tope: pegar
deshace pegar
deshace borrar
deshace escribir
```

append apila, pop desapila

Las dos trabajan en el final de la lista y cuestan $\Theta(1)$, amortizado al apilar. `acciones[-1]` es la última casilla: el tope. Con la cima al principio, `insert(0, x)` y `pop(0)` harían la misma pila a $\Theta(n)$ por operación.

Lo que cuesta cada operación de deque

Operación	Tiempo	Por qué
`append`, `appendleft`	$\Theta(1)$	toca el bloque del extremo
`pop`, `popleft`	$\Theta(1)$	igual, por el otro lado
`d[0]`, `d[-1]`	$\Theta(1)$	los extremos están a mano
`d[i]` en el medio	$O(n)$	recorre bloques hasta llegar
`x in d`	$O(n)$	compara de una en una
`len(d)`	$\Theta(1)$	el tamaño está guardado

Espacio: $\Theta(n)$ en bloques, y $O(1)$ adicional por operación. Fuente: documentación de `collections.deque`. Para acceder por posición en el medio, `list`.

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

Contar palabras

Con la entrada `sol mar sol luna mar sol`, el `map` de C++ imprimió `luna 1`, `mar 2`, `sol 3`. La versión con `dict`:

```
veces = {}
for linea in sys.stdin:
    for pal in linea.split():
        if pal in veces:
            veces[pal] = veces[pal] + 1
        else:
            veces[pal] = 1
for clave in veces:             # orden de llegada
    print(clave, veces[clave])
```

Pregunta

¿En qué orden salen las palabras?

Contar palabras

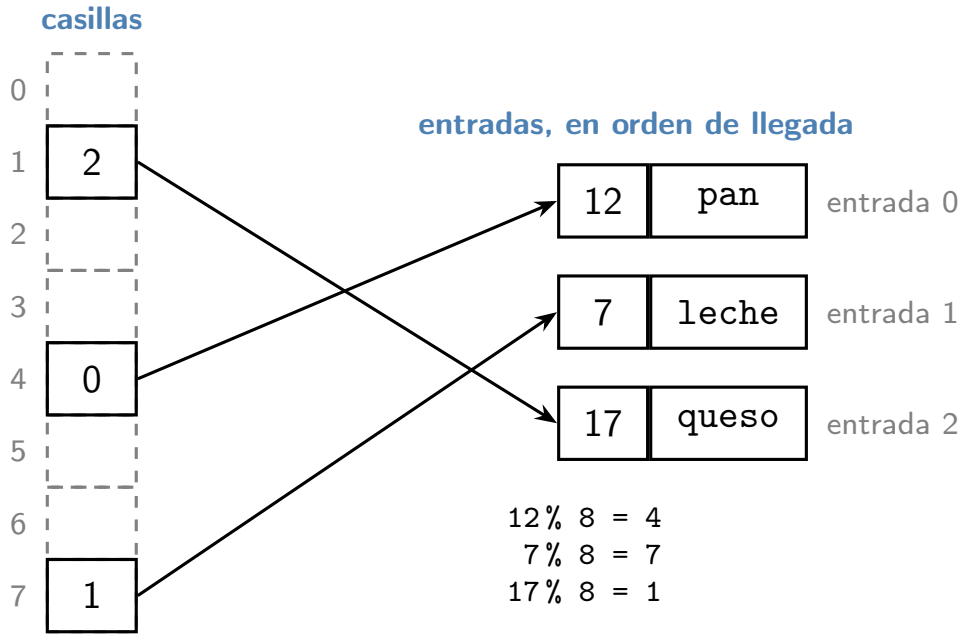
```
print("--- en orden de clave")
for clave in sorted(veces):
    print(clave, veces[clave])
```

```
sol 3
mar 2
luna 1
--- en orden de clave
luna 1
mar 2
sol 3
```

Respuesta

En el orden en que llegaron: `sol`, `mar`, `luna`. El `dict` es una tabla hash, como `unordered_map`, y no un árbol como `map`. Para recorrer en orden de clave, `sorted(veces)` copia las claves y las ordena: $\Theta(n \log n)$.

El `dict` por dentro



Casillas y entradas (CLRS, capítulo 11)

La casilla sale del *hash* de la clave; con enteros pequeños, `hash(k)` es `k`. La casilla guarda el número de la entrada, y las entradas quedan en orden de llegada: por eso el recorrido sale en ese orden. Si la casilla está ocupada, se prueba otra de la misma tabla (CLRS, §11.4).

dict en uso

```
precios = {"pan": 1800, "leche": 4200}
precios["queso"] = 9500          # inserta
precios["pan"] = 2000            # ya estaba: cambia
print(precios)
print(precios["leche"], precios.get("cafe", 0))
print("queso" in precios, "cafe" in precios)
del precios["leche"]
print(precios, len(precios))
```

```
{'pan': 2000, 'leche': 4200, 'queso': 9500}
4200 0
True False
{'pan': 2000, 'queso': 9500} 2
```

`precios["cafe"]` con una clave que no está lanza `KeyError`; el corchete de `map` la habría creado con 0. `get` devuelve el valor por omisión sin insertar.

```
>>> {"Carlos":15, "Maria":12, "Alvaro":13}
{'Carlos': 15, 'Maria': 12, 'Alvaro': 13}
>>> a = {"Carlos":15, "Maria":12, "Alvaro":13}
>>> a
{'Carlos': 15, 'Maria': 12, 'Alvaro': 13}
>>> a["Carlos"]
15
>>> a["Pepe"]
Traceback (most recent call last):
  File "<python-input-9>", line 1, in <module>
    a["Pepe"]
    ~^^^^^^^^
KeyError: 'Pepe'
>>> a.get("Carlos",0)
15
>>> a.get("Pepe",0)
0
```

La lista de asistencia

Pregunta

Con los códigos `42 17 88 17 5 63 42 30`, el `set` de C++ imprimió `5 17 30 42 63 88`. En Python, `set()` y `add` con los mismos códigos. ¿Qué imprime `print`?

La lista de asistencia

```
llegan = [42, 17, 88, 17, 5, 63, 42, 30]
vinieron = set()
for codigo in llegan:
    vinieron.add(codigo)
print(vinieron, len(vinieron))
print(17 in vinieron, 18 in vinieron)
vinieron.discard(17)
print(sorted(vinieron))
```

```
{5, 42, 17, 88, 30, 63} 6
True False
[5, 30, 42, 63, 88]
```

Respuesta

Los repetidos no entran, y el orden es el de las casillas: el `set` de Python es una tabla hash sin valores, no un árbol. No tiene `lower_bound`; el orden sale de `sorted`, en $\Theta(n \log n)$.

Lo que cuesta cada operación de `dict` y `set`

Operación	Tiempo	Por qué
`d[k]`, `d[k]=v`, `del d[k]`	$O(1)$ prom.	se calcula la casilla y se prueban pocas
`k in d`, `s.add(x)`, `x in s`	$O(1)$ prom.	lo mismo
las anteriores, peor caso	$O(n)$	todas en la misma casilla
recorrer	$\Theta(n)$	pasa por todas las entradas
`sorted(d)`	$\Theta(n \log n)$	copia las claves y las ordena

Espacio: $\Theta(n)$ entre casillas y entradas; `sorted` pide $\Theta(n)$ adicional.

Las claves no cambian

Una `list` no puede ser clave de un `dict` ni elemento de un `set`: Python lanza `TypeError`, con el aviso `unhashable type: 'list'`.

```
>>> {(1,2,3,4),(1,2,3)}
{(1, 2, 3, 4), (1, 2, 3)}
>>> {[1,2,4,5],[1,2,4,5]}
Traceback (most recent call last):
  File "<python-input-35>", line 1, in <module>
    {[1,2,4,5],[1,2,4,5]}
TypeError: cannot use 'list' as a set element (unhashable type: 'list')
>>>
```

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

Un montículo de mínimos sobre una `list`

El módulo `heapq`

No es una clase: son funciones que trabajan sobre una `list` cualquiera. La lista `h` es un montículo de mínimos si `h[k] <= h[2*k+1]` y `h[k] <= h[2*k+2]` para toda k con hijos; el menor queda en `h[0]`.

`priority_queue<int> pq`	`h = []`
`pq.push(x)`	`heapq.heappush(h, x)`
`pq.top()`	`h[0]`
`pq.top(); pq.pop()`	`x = heapq.heappop(h)`

La diferencia que importa: `priority_queue` da el mayor y `heapq`, el menor.

heappush en uso

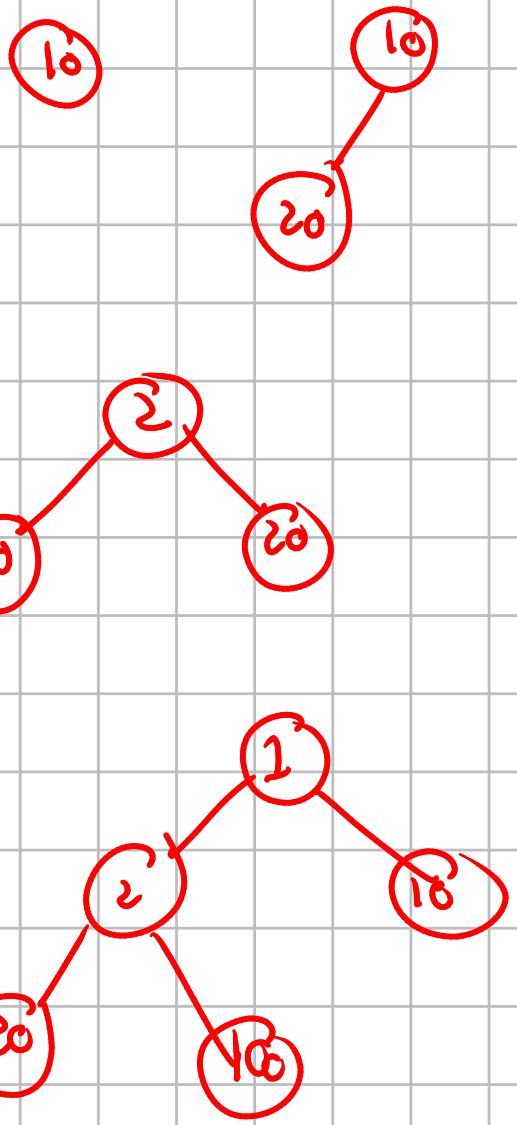
```
h = []
for k in [30, 12, 45, 7, 21]:
    heapq.heappush(h, k)
    print(h)
print("menor:", h[0])
x = heapq.heappop(h)
print("sale", x, "queda", h)
```

```
[30]
[12, 30]
[12, 30, 45]
[7, 12, 45, 30]
[7, 12, 45, 30, 21]
menor: 7
sale 7 queda [12, 21, 45, 30]
```

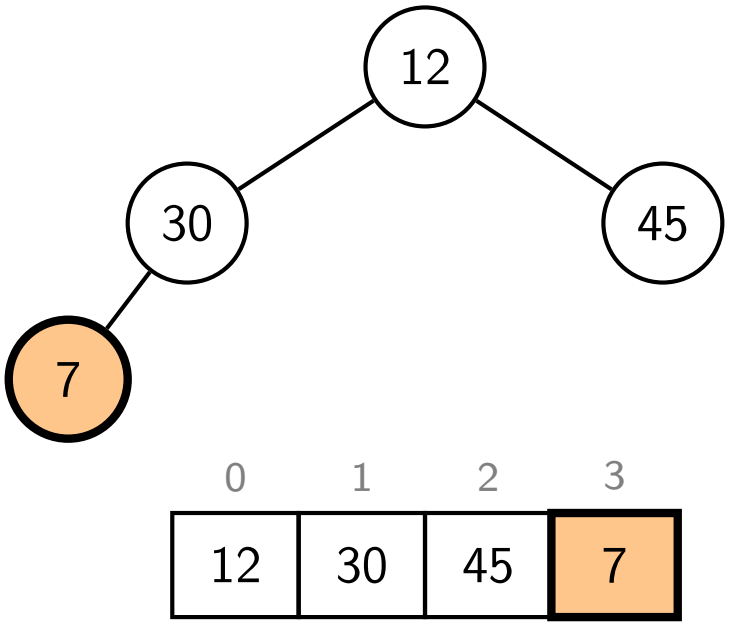
Pregunta

¿Por qué el 7 quedó en `h[0]` y el 30 bajó a `h[3]`?

```
>>> import heapq
>>> l = []
>>> heapq.heappush(l,10)
>>> l
[10]
>>> heapq.heappush(l,20)
>>> l
[10, 20]
>>> heapq.heappush(l,2)
>>> l
[2, 20, 10]
>>> heapq.heappush(l,1)
>>> l
[1, 2, 10, 20]
>>> heapq.heappush(l,100)
>>> l
[1, 2, 10, 20, 100]
>>>
```

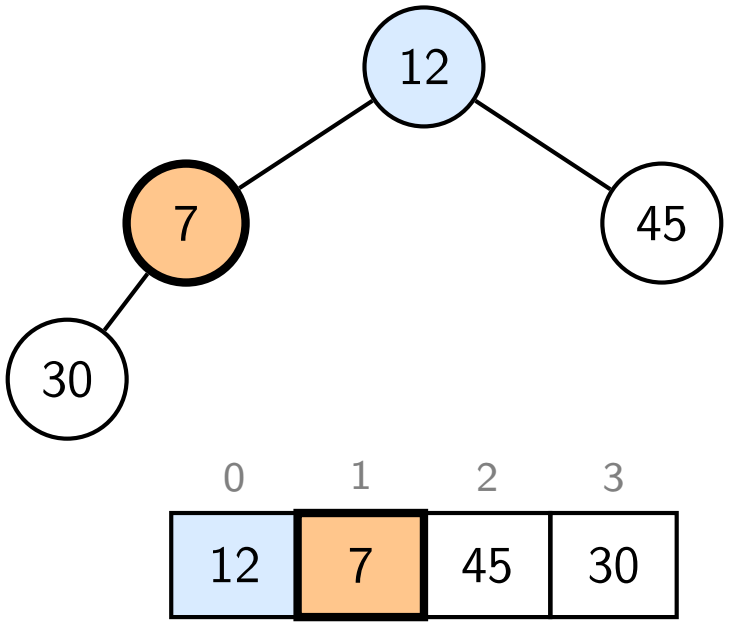


`heappush(h, 7)`, paso a paso



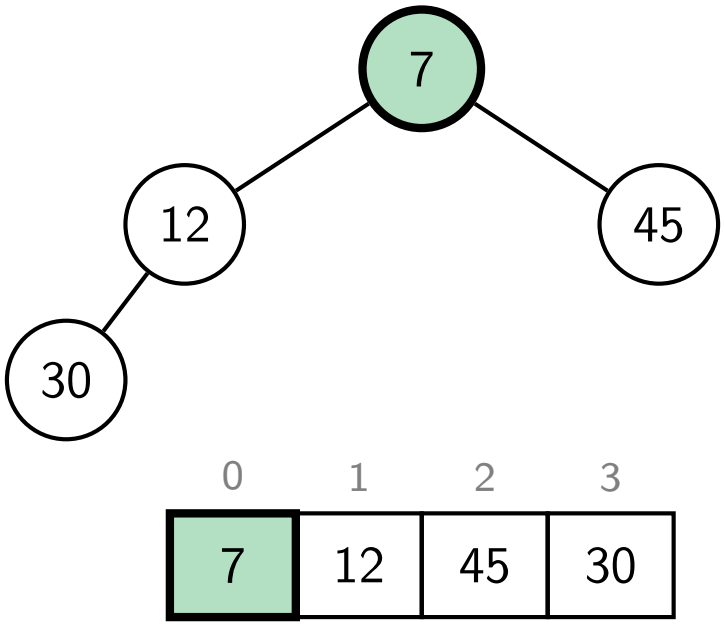
El 7 entra en `h[3]`, la primera casilla libre. Su padre es `h[1]` = 30.

`heappush(h, 7)`, paso a paso



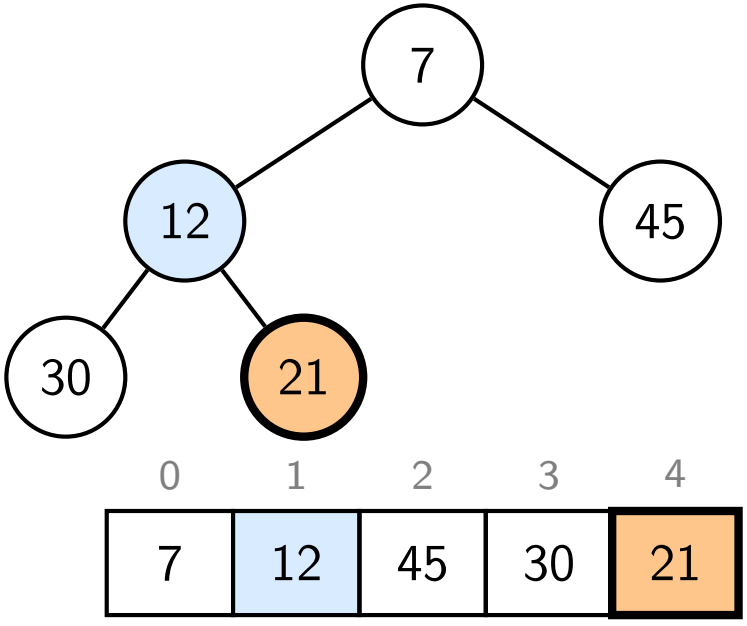
$7 < 30$: se intercambian y el 7 queda en `h[1]`. Su padre es `h[0]` = 12.

`heappush(h, 7)`, paso a paso



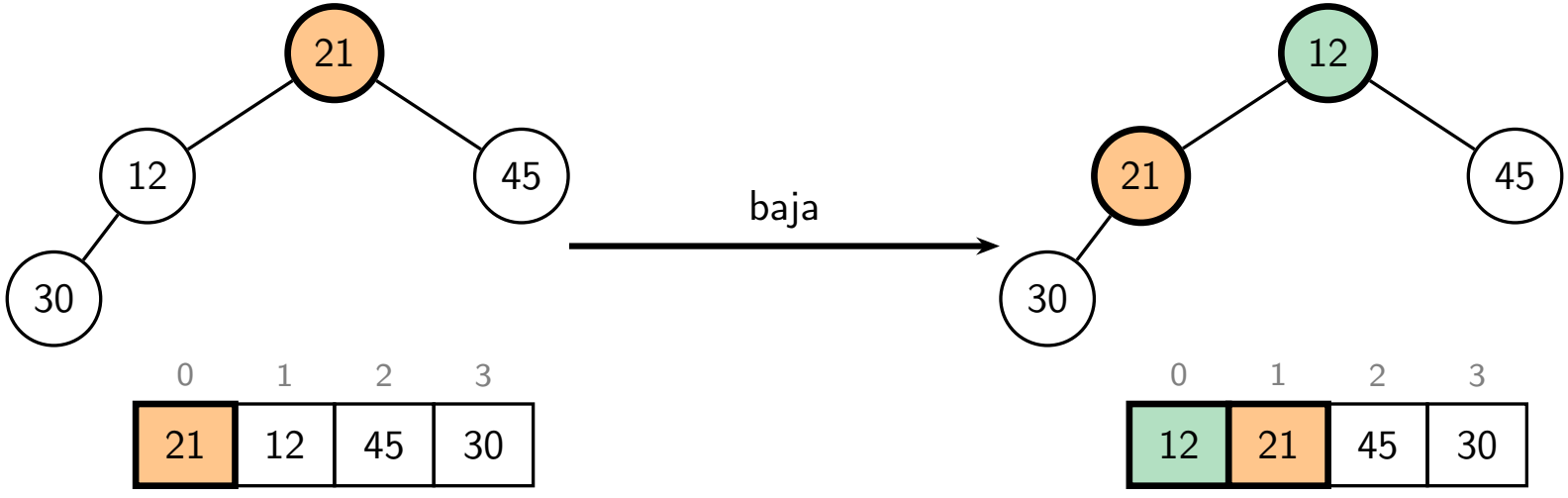
$7 < 12$: el 7 llega a la raíz y se detiene. Dos intercambios, uno por nivel: $O(\log n)$.

`heappush(h, 21)`, paso a paso



El 21 entra en `h[4]`; su padre es `h[1]` = 12, y como $21 > 12$, no sube.

heappop: sale el menor



El espejo de `extraerMax`

Sale el 7 de `h[0]`; el 21, la última clave, pasa a la raíz y baja hacia el *menor* de sus hijos, el 12. En `h[1]` su hijo es el 30, mayor, y se detiene: 12 21 45 30.

El mayor primero

```
m = []
for k in [30, 12, 45, 7, 21]:
    heapq.heappush(m, -k)
print("mayor:", -m[0])
```

```
mayor: 45
```

Si $a > b$, entonces $-a < -b$: el menor de los negados es el mayor de los originales. Se apila $-k$ y se lee $-$`m[0]`.

Un error común

Apilar las claves sin negar y esperar el mayor en `h[0]`: ahí queda el menor.

Pedidos con urgencia

```
pedidos = []
heapq.heappush(pedidos, (-2, 1, "cafe"))
heapq.heappush(pedidos, (-5, 2, "almuerzo"))
heapq.heappush(pedidos, (-5, 3, "postre"))
heapq.heappush(pedidos, (-1, 4, "agua"))
while len(pedidos) > 0:
    urgencia, turno, nombre = heapq.heappop(pedidos)
    print(-urgencia, turno, nombre)
```

```
5 2 almuerzo
5 3 postre
2 1 cafe
1 4 agua
```

Las tuplas se comparan campo por campo

Primero la urgencia negada; si empata, el turno. Sale la urgencia mayor y, entre iguales, el que llegó primero.

Los mismos pedidos con `priority_queue`

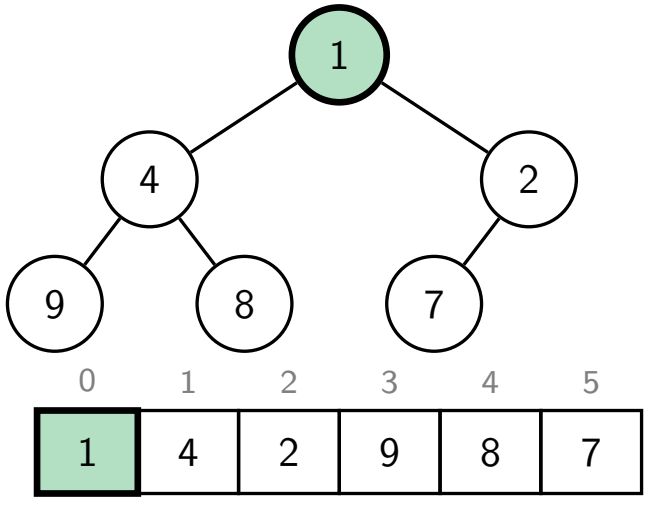
```cpp
struct Pedido {
  int urgencia;
  int turno;
  string nombre;
  // sale primero la urgencia mayor; con la misma, el turno menor
  bool operator<(const Pedido &o) const {
    bool menor;
    if (urgencia != o.urgencia) {
      menor = urgencia < o.urgencia;
    } else {
      menor = turno > o.turno;
    }
    return menor;
  }
};
```

En C++ el orden lo dice `operator<`; en Python, la tupla. Las dos salidas coinciden línea por línea.

De un arreglo cualquiera a montículo

```
datos = [9, 4, 7, 1, 8, 2]
heapq.heapify(datos)
print(datos)
```

```
[1, 4, 2, 9, 8, 7]
```



`heapify` baja cada casilla con hijos, de `h[2]` a `h[0]`: la construcción de abajo arriba de heapsort, en $\Theta(n)$ (CLRS, §6.3).

Lo que cuesta cada operación de `heapq`

Operación	Tiempo	Por qué
`heappush(h, x)`	$O(\log n)$	entra al final y sube por un camino
`heappop(h)`	$O(\log n)$	la última pasa a la raíz y baja
`h[0]`	$\Theta(1)$	el menor está en la raíz
`heapify(l)`	$\Theta(n)$	baja cada casilla con hijos; casi todas están cerca de las hojas
n veces `heappop`	$\Theta(n \log n)$	heapsort con el menor primero

Espacio: la misma `list`, $\Theta(n)$; las operaciones usan $O(1)$ adicional.

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

Sin punteros, sin `new`, sin `delete`

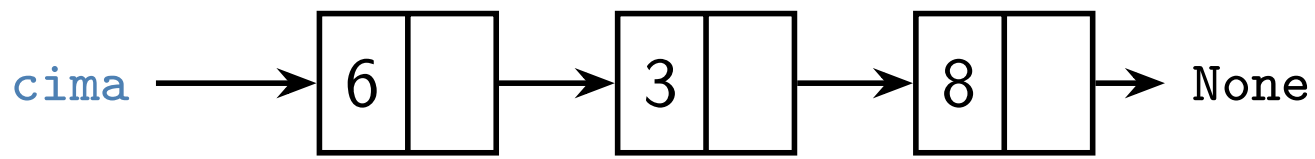
Pregunta

En C++ un nodo guarda `Nodo *siguiente`, se crea con `new` y se libera con `delete`. Python no tiene ninguna de las tres cosas. ¿Cómo se enlazan dos nodos?

Sin punteros, sin `new`, sin `delete`

Respuesta

Toda variable de Python es ya una referencia, como lo era cada casilla de la `list`. El campo `siguiente` guarda la referencia al otro nodo, `None` hace de `NULL`, y el intérprete libera el nodo al que ya nadie apunta.



El nodo

```
struct Nodo {
  Elemento dato;
  Nodo *siguiente;
};
```

```
class Nodo:
    def __init__(self, dato, siguiente):
        self.dato = dato
        self.siguiente = siguiente
```

Una clase con dos campos

`__init__` es el constructor y `self` es el objeto que se construye, como `this`. Los campos se crean al asignarlos. `Nodo(6, None)` hace lo que `new Nodo` con sus dos asignaciones.

apilar y desapilar

```
void apilar(Elemento e) {
  Nodo *nuevo = new Nodo;
  nuevo->dato = e;
  nuevo->siguiente = cima;
  cima = nuevo;
  n = n + 1;
}

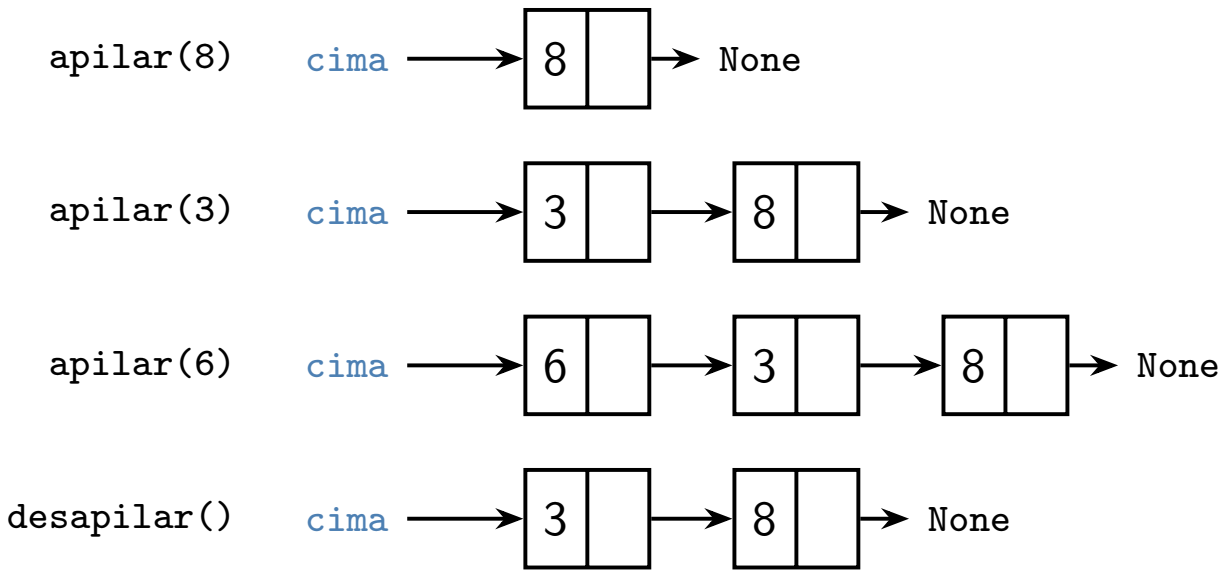
// exige !vacia()
void desapilar() {
  assert(!vacia());
  Nodo *muerto = cima;
  cima = cima->siguiente;
  delete muerto;
  n = n - 1;
}
```

```
    def apilar(self, e):
        nuevo = Nodo(e, self.cima)
        self.cima = nuevo
        self.n = self.n + 1

    # exige not vacia()
    def desapilar(self):
        assert not self.vacia()
        self.cima =
        ↪  self.cima.siguiente
        self.n = self.n - 1
```

En Python, `desapilar` solo mueve `cima`: el nodo viejo queda sin referencias y el intérprete lo libera.

Apilar 8, 3 y 6, y desapilar



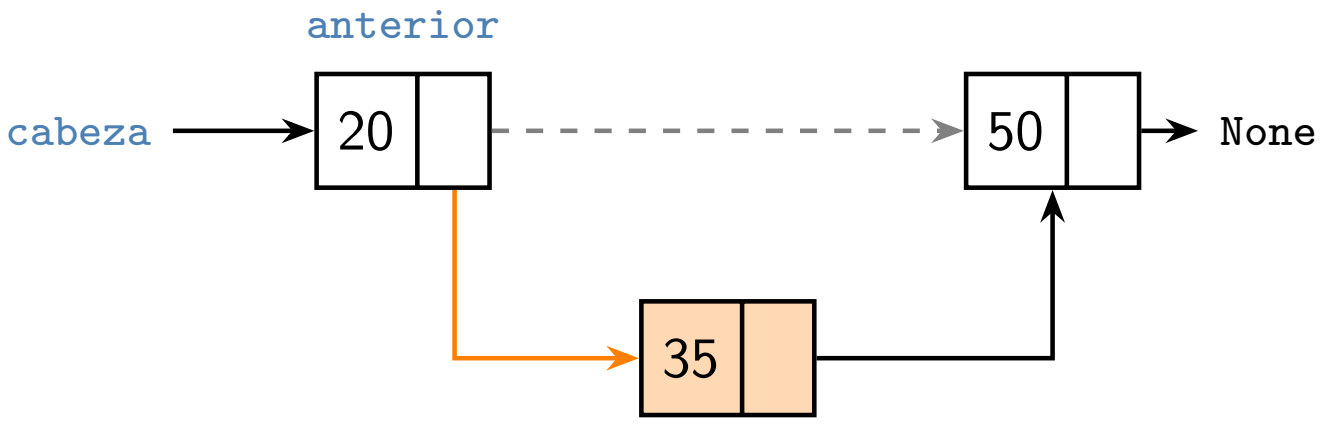
Cada `apilar` crea un nodo que apunta a la cima vieja; `desapilar` pasa la cima al siguiente. El 6 queda sin referencias.

La lista enlazada: insertar

```
    # devuelve el nodo de la posicion p; exige 0 <= p < n
    def nodo_en(self, p):
        actual = self.cabeza
        i = 0
        while i < p:
            actual = actual.siguiente
            i = i + 1
        return actual

    # exige 0 <= p <= tamano()
    def insertar(self, p, e):
        assert 0 <= p and p <= self.n
        if p == 0:
            self.cabeza = Nodo(e, self.cabeza)
        else:
            anterior = self.nodo_en(p - 1)
            anterior.siguiente = Nodo(e, anterior.siguiente)
        self.n = self.n + 1
```

`insertar(1, 35)` sobre 20 50



1. `Nodo(35, anterior.siguiente)`: el nuevo apunta al 50.
2. `anterior.siguiente = ...`: el 20 deja el 50 y apunta al 35.

El orden importa: si el 20 apuntara primero al nuevo, se perdería la referencia al 50.

La lista y la pila en uso

```
p = Pila()
p.apilar(8)
p.apilar(3)
p.apilar(6)
print("tope:", p.tope(), " tamano:",
↪  p.tamano())
p.desapilar()
print("tope:", p.tope(), " tamano:",
↪  p.tamano())

lista = Lista()
lista.insertar(0, 20)
lista.insertar(1, 50)
lista.insertar(1, 35)
lista.insertar(0, 10)
lista.imprimir()
lista.eliminar(2)
lista.imprimir()
print("obtener(1):", lista.obtener(1))
```

```
tope: 6  tamano: 3
tope: 3  tamano: 2
 10 20 35 50   n = 4
 10 20 50   n = 3
obtener(1): 20
```

Lo que cuesta la pila y la lista sobre nodos

Operación	Tiempo	Por qué
`apilar`, `desapilar`, `tope`	$\Theta(1)$	solo se toca la cima
`tamano`, `vacia`	$\Theta(1)$	`n` está guardado
`nodo_en(p)`	$\Theta(p)$	camina p nodos desde la cabeza
`insertar(p, e)`, `eliminar(p)`	$O(p)$	camina hasta el anterior y reenlaza en $\Theta(1)$
`obtener(p)`	$O(p)$	lo mismo que `nodo_en`

Espacio: $\Theta(n)$, un nodo por elemento con su dato y una referencia; las operaciones usan $O(1)$ adicional (CLRS, §10.2).

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

Cada problema pide unas operaciones



Pregunta

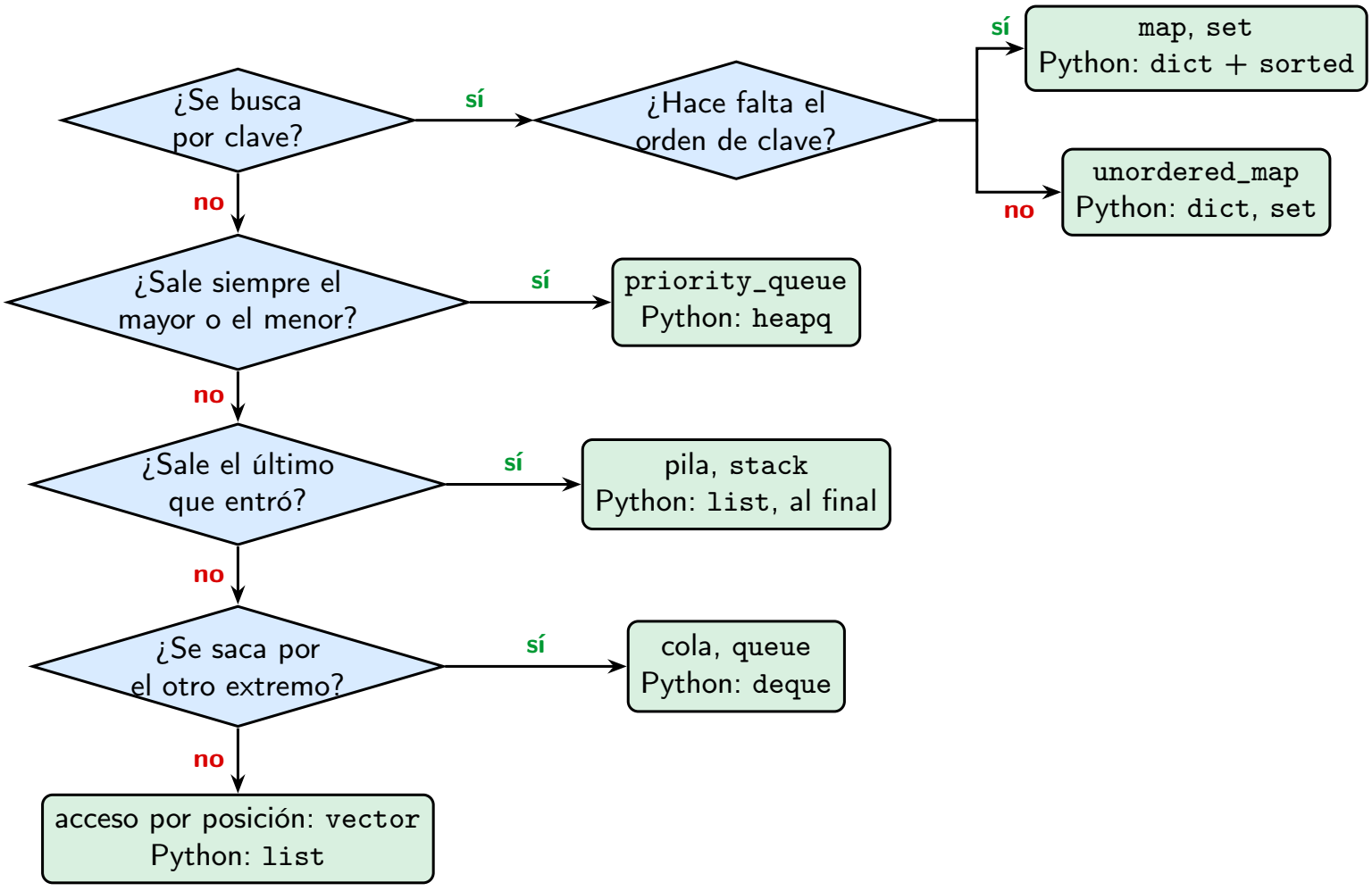
Cajones numerados, una pila de platos, una carpeta con pestañas, una banda transportadora y una pirámide de naranjas. ¿Qué estructura del curso es cada compartimiento?

Cada problema pide unas operaciones

Respuesta

- Cajones numerados: acceso por posición, `list` o `vector`.
- Platos: se saca el último que llegó, una pila.
- Carpeta con pestañas: se busca por clave, `dict` o `map`.
- Banda: sale el primero que llegó, `deque` o `queue`.
- Naranjas: arriba el mejor, `heapq` o `priority_queue`.

Elegir por las operaciones



Las equivalencias, en una tabla

C++	Python	Lo barato y su costo
`vector`	`list`	posición, $\Theta(1)$; el final, $O(1)$ amort.
`deque`, `queue`	`deque`	los dos extremos, $\Theta(1)$
`stack`	`list`	el final, $O(1)$ amort.
`unordered_map`	`dict`	buscar por clave, $O(1)$ prom.
`map`, `set`	`dict`, `set` y `sorted`	clave en orden; Python ordena aparte
`priority_queue`	`heapq`	mayor en C++, menor en Python, $O(\log n)$
lista sobre nodos	clase `Nodo`	insertar tras un nodo dado, $\Theta(1)$

La biblioteca estándar de Python no trae un árbol binario de búsqueda balanceado: el orden de clave sale de `sorted`.

¿Qué estructura?

Pregunta

Un navegador guarda las páginas visitadas. El botón *atrás* vuelve a la última, y al abrir una página nueva esta se agrega. ¿Qué estructura de Python usa, y cuánto cuesta cada botón?

¿Qué estructura?

Respuesta

Se saca la última que entró: una pila. En Python, una `list` con `append` al abrir y `pop()` al ir atrás. Las dos trabajan en el final: $\Theta(1)$, la primera amortizada.

Si además se quiere saber al instante si una página ya se visitó, la pila no basta: `x in l` es $O(n)$. Se agrega un `set` al lado, con `in` en $O(1)$ promedio.

¿Qué estructura?

Pregunta

Una votación recibe un millón de votos, cada uno con el nombre de un candidato. Al cerrar, hay que imprimir cada candidato con sus votos, en orden alfabético. ¿Qué estructura de Python, y cuánto cuesta todo?

¿Qué estructura?

Respuesta

Se busca por clave y se suma: un `dict` de candidato a votos, $O(1)$ promedio por voto, $O(n)$ para los n votos. Al cerrar, `sorted` sobre los c candidatos: $\Theta(c \log c)$. En total $O(n + c \log c)$.

En C++, un `map<string, int>` ya entrega el orden, pero cada voto cuesta $O(\log c)$: $O(n \log c)$ en total.

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

Propuestos para la casa

1. **La cola con dos pilas.** Implementen una cola con dos `list` usadas como pilas, sin `pop(0)`. ¿Cuánto cuesta atender a n personas?
2. **Los k menores.** Dada una `list` de n números, impriman los k menores en orden con `heapify` y `heappop`. Den el costo en función de n y k.
3. **La cola sobre nodos.** Escriban en Python la cola de la implementación enlazada, con `frente` y `final`, y comparen su costo con el de `deque`.

Pistas para los propuestos

1. Una pila recibe y la otra entrega; cuando la que entrega está vacía, se pasan todos de la una a la otra. Cada persona se mueve una sola vez: $\Theta(n)$ en total, $O(1)$ amortizado por operación.
2. `heapify` cuesta $\Theta(n)$ y cada `heappop`, $O(\log n)$: $O(n + k \log n)$.
3. `encolar` enlaza después de `final` y `desencolar` mueve `frente`: las dos en $\Theta(1)$.

Contenido

1 Repaso: heapsort y la cola de prioridad

2 `list`: el vector de Python

3 `deque`: colas y pilas

4 `dict` y `set`: tablas hash

5 `heapq`: el montículo de Python

6 Nodos en Python: la pila y la lista enlazada

7 Qué estructura elegir

8 Para practicar en casa

9 Cierre

Resumen

- `list` es el `vector`: un arreglo de referencias que crece solo, con acceso y `append` baratos y `pop(0)` en $\Theta(n)$. Como pila, con la cima al final.
- `deque` da los dos extremos en $\Theta(1)$: es la cola.
- `dict` y `set` son tablas hash: $O(1)$ en promedio, sin orden de clave; para el orden, `sorted`.
- `heapq` es un montículo de mínimos sobre una `list`; para el mayor, se niegan las claves.
- Los nodos se enlazan con referencias, y `None` hace de `NULL`.

Referencias

- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 6: §6.3 a §6.5. Capítulo 10: §10.1 pilas y colas, §10.2 listas enlazadas. Capítulo 11, tablas hash: §11.4, direccionamiento abierto. Capítulo 12, árboles binarios de búsqueda. Capítulo 16: §16.4, tablas dinámicas.
- Python. *Data Structures* (tutorial, sección 5): `https://docs.python.org/3/tutorial/datastructures.html`.
- Python. *collections*: `https://docs.python.org/3/library/collections.html`.
- Python. *heapq*: `https://docs.python.org/3/library/heapq.html`.
- Python. *Time complexity of operations on built-in types*: `https://docs.python.org/3/builtins/time-complexity.html` (antes `https://wiki.python.org/moin/TimeComplexity`).

1. Complejidad

Conteo de operaciones, operaciones fundamentales (sumas, restas asignaciones) O(1), ciclos (siempre hay una verificacion de salida)

Punteros: modelo de memoria: Pila, Heap, Codigo/Datos, pila tenemos lo que tienen las funciones y las variables locales, Heap globales, objetos, arrelgos, Los arreglos es un puntero a una reservar de memoria de elementos, una matriz un arreglo de punteros

Implementacion: TAD (Objetos para representar cosas, Racionales p/q en lugar un decimal), Vectores, Listas, Pilas, Colas, Cola doble, Maps (ordenado y el no ordenado), Tablas hash, monticulos, arboles ABB; y colas de prioridad

STL: Las mismas estructuras, iteradores para recorrer tener cuidado cuando cambia el tamaño (colgante), begin(), end(). Algorithm.